Project 2 – Acme Analytics Linux Platform

Day 10 · Phase 4: Step 4.5 concept – cgroups, cache and swap, container networking

Muhammad Bilal Javed · 1 November 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	gw01 (PID pool check) · concept day – the cgroups lab on app01 and stor01 is next
Status	Phase 4 🟡 – Steps 4.1–4.4 ✅ · Step 4.5 🟡 concept ✅ (cgroups, nice vs limits, throttling, MemoryHigh/MemoryMax, cache and swap, Docker/Kubernetes limits, who creates the box, container networking) – hands-on lab next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot or diagram. Terms are explained in the glossary. Previous day: Day 9.

---

1. Warm-up – the PID pool on gw01

```
cat /proc/sys/kernel/pid_max      # 4194304 — size of the PID pool
cat /proc/loadavg                 # 0.01 0.07 0.03 1/238 1823
ps -e --no-headers | wc -l        # 209 processes
```

`/proc/loadavg` = load over 1 / 5 / 15 minutes (almost zero – gw01 is idle) · **1/238** = 1 task running out of 238 (threads included) · **1823** = the PID handed out most recently. `ps -e` counts **processes** only (209), so it is lower than 238.

---

2. Step 4.5 – the concept: cgroups

💡 **In plain words:** `nice` decides **who goes first**; a **cgroup** decides **how much at most**. Every service runs in its own **box**, and a limit on the box is a hard ceiling for every process inside it – so one bad job cannot freeze the whole server.

2.1 nice vs cgroup limits

	nice	cgroup limits
works on	each **process**	the whole **box** (service)
means	priority – who goes first, no ceiling	hard ceiling – how much at most
in a unit file	`Nice=19` = start value for every process of the service (children inherit it; `renice -n 10 -p <PID>` changes one)	`CPUQuota=` · `MemoryHigh=` · `MemoryMax=` · `TasksMax=` – shared by all processes in the box

2.2 Who creates the box

- **Linux service: systemd** reads the unit file, starts the service, creates its box (`/sys/fs/cgroup/system.slice/<name>.service/`) and writes the limits. **One service = one box.**
- **Docker:** `dockerd` → `containerd` (the manager) → `runc` (the builder: **namespaces** = isolation, **cgroup** = limits).
- **Kubernetes:** `kubelet` → `containerd` → `runc` – the same Linux cgroups underneath.

2.3 What each limit does when it is exceeded

Limit	Over the limit
`CPUQuota=30%`	**throttled** – the box runs only 30 ms of every 100 ms, like mobile data after the monthly limit: slower, never killed. Other services in their own boxes are **not** slowed.
`MemoryHigh=150M` (soft)	the box is **slowed down**: step 1 – the **cache** (copies of files kept in RAM; originals on disk) is deleted from RAM · step 2 – idle **program memory** is moved from RAM to **swap**, a reserved area **on the disk** (Ubuntu `/swap.img`, Windows `pagefile.sys`). Normal files on the disk are never touched; nothing is killed.
`MemoryMax=200M` (hard)	the kernel's **OOM killer** stops a process **in this box only**
`TasksMax=20`	a new fork **fails** inside this box only ("Resource temporarily unavailable") – protects the PID pool
`MemoryMin=` / `MemoryLow=`	the opposite: a **guaranteed / protected** minimum for the box

2.4 The archive worker as an example (stor01)

```
        [Service]
        ExecStart=/usr/local/bin/archive-worker.sh
        Nice=19                 # priority — per process
        IOSchedulingClass=idle
        CPUQuota=30%            # limits — for the whole box
        MemoryHigh=150M
        MemoryMax=200M
        TasksMax=20
```

Set them in the unit file (`daemon-reload` + `restart`) or with `sudo systemctl set-property archive-worker CPUQuota=30% MemoryMax=200M`; check with `systemctl show archive-worker -p Nice -p CPUQuotaPerSecUSec -p MemoryMax -p TasksMax`.

📸 **Diagram:**

cgroups – how a service gets its box and its limits

Project 2 · Phase 4 · Step 4.5 · example: archive-worker.service on stor01

① unit file (what I write)

```
[Service]
ExecStart=/usr/local/bin/archive-worker.sh

# priority – per PROCESS
Nice=19
IOSchedulingClass=idle

# limits – for the whole BOX
CPUQuota=30%
MemoryHigh=150M
MemoryMax=200M
TasksMax=20
```

reads

② systemd (PID 1)

starts the service
creates the box · writes the limits

creates + applies limits

③ the box (cgroup)

`/sys/fs/cgroup/system.slice/archive-worker.service/`

archive-worker.sh	tar czf …	gzip
PID 812 · nice 19	PID 4211 · nice 10 (renice)	PID 4212 · nice 19 (child)

one service = one box · all its processes and their children live here · Nice=19 is only the START value
each process can get its own nice (renice -n 10 -p 4211) · the limits are shared by the whole box

`CPUQuota 30%` over → THROTTLED slowed down: 30 ms of every 100 ms	`MemoryHigh 150M` over → slowed down soft limit · drop cache / swap · no kill
`MemoryMax 200M` over → OOM KILLER stops a process in THIS box only	`TasksMax 20` over → new fork FAILS "Resource temporarily unavailable"

④ other services = other boxes

`sshd.service` · `nightly-backup.service` · `glpi` …
never lose memory or CPU because of this box
→ one bad job cannot freeze the whole server

⑤ real example – compressing uploads at 14:00 while users work

1. **admin queues a big job** — `sudo archive-enqueue /data/app/uploads`
2. **the worker starts tar + gzip** — children inherit nice 19 and live in the SAME box
3. **gzip wants 100 % CPU** — → throttled to 30 % · job ~3× slower, but finishes
4. **other services NOT slowed** — sshd, GLPI, backup = own boxes · use the free 70 %
5. **gzip memory explodes > 200M** — → OOM killer stops gzip only · job FAILED, trap cleans .part
6. **urgent? admin speeds tar up** — `sudo renice -n 10 -p 4211 (box limit stays)`

Without the box: one compression job could slow the whole server for everyone. With it: the job is slower, everyone else is fine.

⑥ what MemoryHigh does – first drop CACHE, then use SWAP (same idea on Linux and Windows)

RAM before	process memory (really in use)	cache = files kept in RAM (original on disk)	free
RAM after step 1	process memory (really in use)	cache deleted from RAM → room again	
RAM after step 2	active memory	RAM → SWAP	idle program memory moved from RAM to swap – slow, nothing killed

Swap = a reserved area ON THE DISK (Ubuntu: /swap.img) used as extra slow RAM · check: free -h (RAM + swap) · swapon --show (where) · Windows: pagefile.sys

⑦ what "throttled" means – CPUQuota=30% (like mobile data after the limit: slower, not stopped)

30 ms run	70 ms paused	30 ms run	70 ms paused	30 ms run	70 ms paused	30 ms run	70 ms paused

every 100 ms the box may run only 30 ms → the job still finishes, about 3× slower · top shows ~30 %

⑧ the same cgroup limits in Docker and Kubernetes

	CPU ceiling (→ throttled)	memory ceiling (→ killed)	guaranteed minimum
systemd unit	`CPUQuota=30%`	`MemoryMax=200M`	– (`MemoryHigh` = soft limit)
docker run	`--cpus=0.3`	`--memory=200m`	–
docker-compose.yml	`cpus: 0.3`	`mem_limit: 200m`	–
Kubernetes YAML	`limits: cpu: "300m"`	`limits: memory: "200Mi"`	`requests: cpu / memory`

⑨ who creates the box – and the network – in Linux, Docker and Kubernetes

Linux service	**systemd** PID 1 – starts the service	→ **cgroup** one box per service		network: **the host's own network**
Docker	**dockerd** Docker engine	→ **containerd** manages containers	→ **runc** builds the container	→ **namespaces + cgroup** isolation + limits	network: **Docker networking**
Kubernetes	**kubelet** agent on each node	→ **containerd** manages containers	→ **runc** builds the container	→ **namespaces + cgroup** isolation + limits	network: **CNI plugin**

containerd = the manager (images, start/stop) · runc = the builder: namespaces = its own view of processes, network, hostname, files · cgroup = its limits
runc creates an EMPTY network namespace – the IP and connections come from Docker networking (bridge docker0, -p ports) or a Kubernetes CNI plugin (Calico, Flannel)

⑩ final paragraph (corrected) – nice, cgroups, memory, swap

nice sets the priority of each process (-20 highest, 19 lowest) – who goes first, with no ceiling. A cgroup is a box that systemd creates for every service: one service = one box, and all its processes and their children live in it. In the unit file, Nice=19 is only the start value for each process (renice can change one), while CPUQuota, MemoryHigh, MemoryMax and TasksMax are limits shared by the whole box. Over CPUQuota the box is throttled – it runs only e.g. 30 ms of every 100 ms, like mobile data after the limit: slower, never killed – and other services in their own boxes are not slowed. The CPU works only with RAM, so files read from disk are kept as copies in RAM, the cache. Above MemoryHigh (soft limit) the kernel first deletes this cache from RAM – nothing is lost, the originals are on the disk – and then moves idle program memory from RAM into swap, a reserved area on the disk (Ubuntu: /swap.img; Windows: pagefile.sys). Normal files on the disk are never touched; the box only gets slower. If it still reaches MemoryMax (hard wall), the kernel's OOM killer stops a process in this box only. TasksMax caps the number of processes and protects the PID pool. Docker (--memory, --cpus, mem_limit) and Kubernetes (requests = guaranteed, limits = hard ceiling, OOMKilled) use the same cgroups. Check RAM and swap with free -h, where swap lives with swapon --show, and the boxes live with systemd-cgtop. Who creates the box: on Linux systemd; in Docker dockerd → containerd → runc; in Kubernetes kubelet → containerd → runc. containerd manages, runc builds the container with namespaces (isolation) and a cgroup (limits); the IP comes from Docker networking or a CNI plugin.

cgroups – how a service gets its box and its limits

2.5 The same limits in Docker and Kubernetes

	CPU ceiling (→ throttled)	memory ceiling (→ killed)	minimum / soft
systemd unit	`CPUQuota=30%`	`MemoryMax=200M`	`MemoryMin=` guaranteed · `MemoryHigh=` soft
`docker run`	`--cpus=0.3`	`--memory=200m`	`--memory-reservation` (soft only – no guarantee)
docker-compose.yml	`cpus: 0.3`	`mem_limit: 200m`	`deploy.resources.reservations`
Kubernetes YAML	`limits: cpu: "300m"`	`limits: memory: "200Mi"` (→ OOMKilled)	`requests:` = guaranteed, used to choose the node

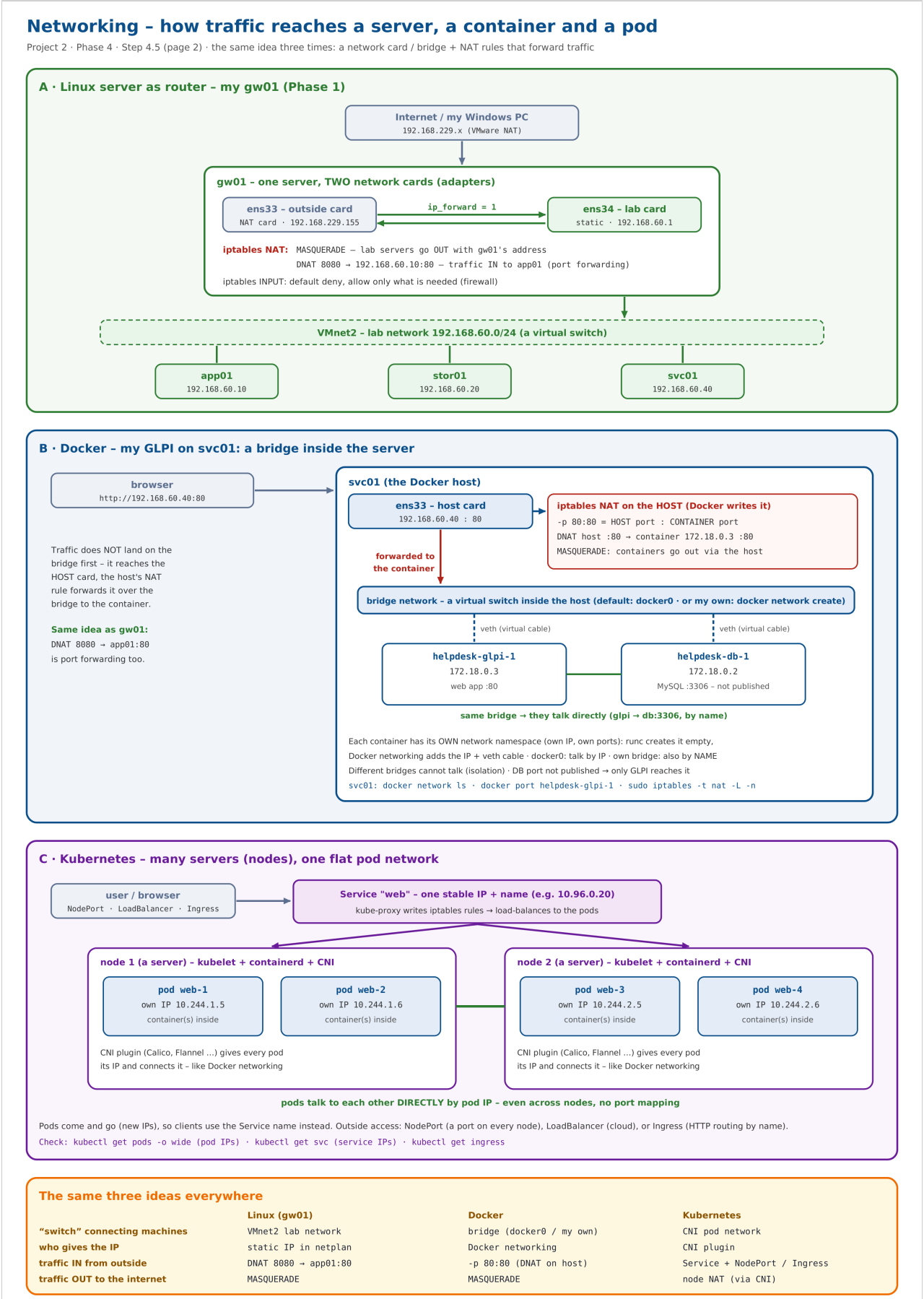
The Dockerfile only builds the image – limits are set when the container runs. `300m` = 300 millicores (0.3 of a core); `200Mi` ≈ 200 MB.

---

3. Container networking – the same idea as gw01

- **Linux (gw01):** two network cards (ens33 outside, ens34 lab), `ip_forward = 1`, **MASQUERADE** for traffic out, **DNAT 8080 → app01:80** for traffic in.
- **Docker (GLPI on svc01):** traffic reaches the **host card** first; the host's iptables NAT rule written by Docker (`-p 80:80` = HOST port : CONTAINER port) forwards it over a **bridge** (default `docker0` or my own `docker network create`) and a **veth** virtual cable to the container's own IP. Containers on the same bridge talk directly (on my own bridge also by name); the MySQL port is not published, so only GLPI reaches it.
- **Kubernetes:** a **CNI plugin** gives every pod its own IP; pods talk directly, even across nodes. A **Service** gives one stable IP/name and **kube-proxy** load-balances to the pods; outside access via NodePort, LoadBalancer or Ingress.

📸 **Diagram:**



Networking – Linux, Docker and Kubernetes

4. Key understanding (in my own words)

nice sets the priority of each process (-20 highest, 19 lowest) – who goes first, with no ceiling. A **cgroup** is a box that **systemd** creates for every service: one service = one box, and all its processes and their children live in it. In the unit file, `Nice=19` is only the start value for each process (`renice` can change one),

while `CPUQuota`, `MemoryHigh`, `MemoryMax` and `TasksMax` are limits shared by the whole box. Over **CPUQuota** the box is **throttled** – it runs only e.g. 30 ms of every 100 ms, like mobile data after the limit: slower, never killed – and other services in their own boxes are not slowed. The CPU works only with **RAM**, so files read from disk are kept as copies in RAM, the **cache**. Above **MemoryHigh** (soft limit) the kernel first deletes this cache from RAM – nothing is lost, the originals are on the disk – and then moves idle program memory from RAM into **swap**, a reserved area on the disk (Ubuntu `/swap.img`, Windows `pagefile.sys`); normal files on the disk are never touched, the box only gets slower. If it still reaches **MemoryMax** (hard wall), the kernel's **OOM killer** stops a process in this box only. **TasksMax** caps the number of processes and protects the PID pool. **Docker** (`--memory`, `--cpus`, `mem_limit`) and **Kubernetes** (requests = guaranteed, limits = hard ceiling, `OOMKilled`) use the same cgroups: on Linux systemd creates the box; in Docker `dockerd → containerd → runc`; in Kubernetes `kubelet → containerd → runc`. containerd manages, runc builds the container with **namespaces** (isolation) and a **cgroup** (limits); the IP comes from Docker networking or a CNI plugin. Traffic to a container always reaches the **host's card** first, and a **NAT rule** forwards it – exactly like my gw01's `DNAT 8080 → app01:80`.

---

5. Next – the cgroups lab

1. **Step 1 (app01):** `free -h` · `swapon --show` · install `stress-ng` · `cat /proc/$$/cgroup` · `systemd-cgls`.
2. **Step 2 (app01):** memory limit – `systemd-run --scope -p MemoryMax=100M stress-ng --vm 1 --vm-bytes 300M` → OOM in the journal.
3. **Step 3 (app01):** CPU limit – `CPUQuota=20%` → `top` and `systemd-cgtop` show ~20 %.
4. **Step 4 (stor01):** permanent limits on `archive-worker.service`.
5. Then: first incident drill · L3 Problem in GLPI · snapshot `50-processes`.